

THE STRUCTURE OF MATHEMATICAL REALITY

Phase III — Chapter 8

Vector Spaces and the Meaning of “Canvas”

Lecture

Self-Study Curriculum

You use linear algebra constantly. You compute covariance matrices, run principal component analysis, fit linear models, multiply weight matrices. This chapter is not about teaching you linear algebra; you already operate it fluently. It is about reframing what you already know so that the structural content becomes visible, because the operational fluency hides a set of choices you are making without noticing, and those choices are the whole subject. Chapter 7 left you with a loose thread: representations act on a *vector space*, and we never said what that space is or where it comes from. Pulling that thread is the business of this chapter, and it ends with your own intuition, that linearity belongs to a system together with its canvas, stated precisely enough to use.

What a vector space actually is

A vector space is a set of objects (called vectors) equipped with two operations, addition and scalar multiplication, satisfying a list of axioms: addition is commutative and associative, there is a zero vector, every vector has a negative, and scalar multiplication distributes over addition in both senses. You do not need to memorize the list. You need to notice what it does *not* say. It does not say the vectors are arrows. It does not say they are columns of numbers. It says only: you can add two of these things and get a third of the same kind, and you can scale one by a number, and these two operations behave sensibly together.

That silence is the entire point. Because the definition only mentions “add” and “scale,” anything you can add and scale, in a way that obeys the axioms, is a vector space, no matter what the objects actually are:

- The functions from $\mathbb{R}$ to $\mathbb{R}$ form a vector space. You can add two functions (add their values at each point) and multiply a function by a scalar. The zero vector is the function that is zero everywhere.
- The solutions of a linear differential equation form a vector space. Add two solutions and you get a solution; scale a solution and you get a solution. (This is exactly why “superposition of solutions” is a phrase you have heard; it is the vector-space structure of the solution set.)
- The set of all possible states of a quantum system forms a vector space. Adding states and scaling them is not a metaphor there; it is the physics.

Worked example: polynomials as vectors. Consider all polynomials of degree at most 3. Add two of them and you get another polynomial of degree at most 3; scale one by a number and you stay in the set. So this is a vector space, and its “vectors” are things like $2x^3 - x + 5$, which look nothing like arrows. You will meet this space again in the exercises, and it will make the abstract definition concrete: a polynomial is a vector because it lives in a set closed under adding and scaling, and that is the only membership requirement there ever is.

This is the reframing the chapter exists to deliver. A vector space is not a *type of object*. It is a *type of structure*. When you learn to recognize the structure, you stop seeing “arrows in the plane,” “columns in a spreadsheet,” “functions,” and “quantum states” as four different subjects and start seeing one structure carrying four kinds of content, exactly the recognition Chapter 7 trained on groups. The tools of linear algebra work on all of them at once because they only ever touch the structure, never the content.

Basis and dimension: the meaning of coordinates

If a vector space is a set you can build things in by adding and scaling, it is natural to ask for the smallest kit of building blocks. A *basis* is a minimal set of vectors from which every other vector can

be built, uniquely, by addition and scaling. The *dimension* of the space is the number of vectors in any basis, and it is the same number no matter which basis you pick, which is already a nontrivial structural fact: the space has a size that does not depend on how you choose to describe it.

Here is the key structural insight, and it is the one to carry out of this section: **a choice of basis is a choice of description, not a fact about the space.** The vector is a real object; its *coordinates* are what you get after you commit to a basis. Change the basis and the coordinates change, but the vector does not move. This is why “change of basis” is one of the most important operations in all of applied mathematics: it is the act of translating between two equally valid descriptions of the same unchanged object.

Worked example: one polynomial, two coordinate systems. The space of polynomials of degree at most 3 has dimension 4. The obvious basis is $\{1, x, x^2, x^3\}$. In that basis, the polynomial $2x^3 - x + 5$ has coordinates $(5, -1, 0, 2)$: five ones, minus one x , no x^2 , two x^3 . But nothing forces that basis. Suppose you care about behavior near $x = 1$, and you use the basis $\{1, (x-1), (x-1)^2, (x-1)^3\}$ instead. The very same polynomial now has different coordinates, the numbers you would get from a Taylor expansion about $x = 1$, because you are describing it by its value and derivatives at 1 rather than by powers of x . The coordinates are completely different; the polynomial is identical. What changed was your description. What did not change was the object, its degree, its roots, its value at any given point. Coordinates are a report about the object relative to a chosen frame, and confusing the report with the object is the single most common source of structural confusion in applied work.

You do this every day without naming it. Choosing which features to record is choosing a basis for the space your data lives in. Raw pixel intensities, or edge-detector responses, or the outputs of a learned encoder, are three bases for “the space of images.” One-hot encoding a category is choosing a basis in which each category is a separate axis. The data does not change when you re-encode it; its coordinates do. Recognizing that your feature choice is a basis choice, not a fact about the phenomenon, is the beginning of controlling it.

What a linear map is

Having said what respects the *objects* (they are vectors) we need to say what respects the *structure between* them. A linear map between vector spaces is a function that preserves both operations: $f(u + v) = f(u) + f(v)$, and $f(cv) = cf(v)$ for any scalar c . These two conditions say the map commutes with adding and scaling: you can add first and then map, or map first and then add, and land in the same place. These are exactly the maps that respect the vector-space structure, and they are the third instance of the theme you met in Chapter 7. The group homomorphism preserved the operation; the linear map preserves addition and scaling; the continuous map of Chapter 9 will preserve nearness. Same idea, different structure preserved.

Every linear map between finite-dimensional vector spaces can be written as a matrix, and this is where matrices come from: a matrix is not a fundamental object but the *coordinate report* of a linear map once you have chosen bases for the input and output spaces. Change the basis and the matrix changes, even though the map, the actual transformation of the space, is unchanged. The same linear map has infinitely many matrix representations, one per choice of basis. So a matrix, like a coordinate vector, is a description relative to a frame, not the thing itself.

This raises a sharp question: if the matrix is an artifact of your basis, what about a linear map is *real*, independent of any description? The answer is: its invariants, and chief among them, its eigenvalues. An eigenvalue is a scaling factor along a special direction that the map merely stretches or shrinks without rotating. Crucially, the eigenvalues are the same no matter which basis you use to write the matrix down. They belong to the map, not to your coordinates. This is

why eigenvalues matter so much: they capture the intrinsic behavior of the transformation, the part that is not an artifact of how you chose to look. You already met this idea in Chapter 5, where the determinant was an invariant under change of basis, and in Chapter 3, where the same eigenvalue was an algebraic, an analytic, and a geometric object at once. Here is the unifying reason: the determinant and the eigenvalues survive change of basis precisely because they are properties of the map, and the basis is only a description of it.

Worked example: two matrices, one map, shared eigenvalues. Imagine a transformation of the plane that stretches everything by a factor of 3 along one line through the origin and leaves a second line through the origin fixed. That is a single geometric operation. If you choose your two basis vectors to point along those two lines, its matrix is diagonal, with 3 and 1 on the diagonal, about as simple as a matrix gets. If instead you choose some skew pair of basis vectors that do not line up with those lines, the matrix comes out full of off-diagonal clutter and looks like a different transformation entirely. But compute the eigenvalues of either matrix and you get 3 and 1 both times. The clutter was yours; the numbers 3 and 1 are the map's. In your own work this is the covariance matrix of a dataset: its eigenvalues are the amounts of variance along its principal directions, and they do not care which coordinate system you happened to record the data in.

Eigenvalues carry intrinsic behavior in dynamics just as they do in geometry, the double life Chapter 3 flagged for them. If a linear map is applied over and over, as when a Markov chain steps forward in time, its largest eigenvalue governs the long-run growth or decay and the corresponding eigenvector is the direction the system settles into; the gap between the largest and the next eigenvalue sets how fast it settles. Same numbers, different reading: geometrically an eigenvalue is a stretch factor, analytically it is a rate. Because the number survives change of basis, it is safe to attach real meaning to it. A coordinate-bound quantity would be meaningless to interpret, because someone else's choice of basis would give a different value; an invariant can be interpreted, because it is the map's own.

The physicist's demand, translated. When a physicist insists that the laws of physics must be the same in every coordinate system, this chapter lets you say exactly what is being demanded. A coordinate system is a basis for the space of states; a change of coordinates is a change of basis; and a law written in coordinates is like a matrix, a description relative to a frame. To require the law to hold in every frame is to require its content to be an *invariant*, a statement about the quantities that survive change of basis (eigenvalues, inner-product relations, the coordinate-free content) rather than about any one frame's numbers. It is the same instinct that makes you trust a dataset's principal directions over its raw recording axes: you are preferring the description-independent content to the description.

Your intuition about “canvas”

You said: “Linearity is not a property of systems but a property of the relationship between a system and its canvas.” You reasoned to that on your own, and it is correct. Here is the precise version, assembled from the pieces above.

A system is described by a state that changes over time. To do anything mathematical with it, you first choose a *representation*: a way of encoding the state as a vector in some space. That space, with its basis, is the canvas. Once you have committed to a canvas, the system's evolution is some map from states to later states, and you can ask whether that map is *linear*, whether it preserves addition and scaling on your chosen canvas. If it is, the evolution is matrix multiplication, and the full arsenal of linear algebra, eigenvalues, decompositions, closed-form solutions, becomes available. If it is not, you are in the harder nonlinear world.

The decisive point is that linearity was a property of the map *on that canvas*, and you chose the

canvas. A different representation of the same system can turn the same evolution from nonlinear to linear or back. Linearity is not a fact about the system; it is a fact about the relationship between the system and the canvas you painted it on, which is exactly what you said.

Worked example: population growth. Represent a population by its size $N(t)$. Exponential growth, $N(t) = N_0 e^{rt}$, is nonlinear in this representation: doubling the elapsed time does not double the population, it squares the growth factor, and the curve bends. Now change the canvas: represent the same population by $\log N(t)$. In this representation the same growth is $\log N(t) = \log N_0 + rt$, a straight line. The system did not change. The bacteria did not consult your notebook. What changed is that on the logarithmic canvas the evolution preserves addition (each unit of time adds a fixed amount), so it is linear, while on the raw-count canvas it does not, so it is not. The linearity lived in the canvas the entire time.

This connects directly to Chapter 5, where you saw that linearity is the *superposition symmetry*: the invariance of a system’s response under adding inputs. A canvas on which a system is linear is a canvas on which superposition holds; the response to A plus B is the response to A plus the response to B. So “find the right canvas” and “find the coordinates in which the superposition symmetry is exact” are the same search. And this is why so much of applied mathematics opens with a change of variables: you are hunting for the canvas on which the problem’s structure, ideally its linearity, becomes visible. The right coordinates are the ones that make the structure of the problem visible, and very often that means the ones that make it linear.

The special case of linearization. Sometimes no canvas makes a system exactly linear, and you settle for a canvas on which it is *approximately* linear in a small region. Linearizing a nonlinear system around an equilibrium point, the small-signal approximation, the Taylor expansion kept to first order, is exactly this. What is being approximated is the true evolution map, replaced by its best linear stand-in near the equilibrium; the structural assumption is that the deviations you care about are small enough that superposition holds to good accuracy. You are painting a curved system onto a flat canvas that touches it at one point, and trusting the flatness only nearby. This is the doorway to Chapter 10, where the state space becomes a canvas with a flow painted on it, and where linearization near equilibria is the standard first move for understanding a nonlinear system’s behavior.

Inner products and the geometry of vector spaces

Everything so far, adding, scaling, bases, linear maps, used no notion of length or angle. A bare vector space has no geometry; it does not know which vectors are “long” or which pairs are “perpendicular.” An *inner product* (the dot product, generalized) is the extra structure that supplies geometry: from it you get the length of a vector, the angle between two vectors, and in particular the notion of *orthogonality*, two vectors being at right angles. But not every vector space comes with a natural inner product, and when there is a choice, the choice matters: different inner products make different pairs of vectors count as perpendicular, and thereby define different geometries on the same underlying space.

A biological example of choosing the inner product. Represent each sample as a vector of gene-expression levels, one component per gene. How similar are two samples? If you take the ordinary dot product of the raw vectors, a sample that expresses everything at high absolute levels looks “large,” and overall magnitude dominates the comparison. If instead you center and normalize each vector first and then take the dot product, you are measuring the cosine of the angle between them, and now two samples count as similar when their *patterns* of relative expression agree, regardless of overall magnitude. Those are two different inner products on the same space, and they encode two different biological notions of “similar.” Neither is correct in the abstract; the right one is the one

that matches the question you are asking. And the choice is silent and constant: every similarity score, every distance inside a clustering, every kernel, is an inner product you have selected, usually without announcing it even to yourself.

Worked example: PCA as a change of canvas. You have a dataset with 100 features, and principal component analysis reports that 95% of the variance is explained by 3 principal components. Translate that into the language of this chapter. The old canvas was the 100-dimensional space whose basis is your measured features, one axis per variable. PCA computes the covariance matrix (which depends on an inner product, on how you decided to measure variance) and finds its eigenvectors: a *new* basis of 100 orthogonal directions, ordered by how much variance the data has along each. That is a change of basis, a rotation to a new canvas, and by itself it loses nothing; it is invertible. The compression happens at the next step, when you keep only the top 3 directions and discard the other 97. The dimension of your working space drops from 100 to 3. What was lost is the 5% of variance in the discarded directions, and with it exact invertibility: you can no longer reconstruct the original data, only its shadow on the 3-dimensional canvas. What was gained is a canvas on which the data’s dominant structure is laid out along a handful of axes, which is a genuine compression, and it is the same compression instinct you flagged in your own thinking: replace a large description by a smaller one that keeps the structure that matters.

The eigenvalues (the singular values, in the usual computation) are the whole point, and they should be read structurally, not as a computation. Each eigenvalue is the amount of variance the data carries along its principal direction. Reading them tells you the *intrinsic dimensionality* of the phenomenon: if three eigenvalues are large and the rest are tiny, the system, whatever its hundred measured features, actually moves in a three-dimensional space, and the other 97 features were largely redundant descriptions of those three degrees of freedom. That is a statement about the system’s structure, and it is exactly the kind of hidden low dimensionality that made Chapter 5’s “powerful operations exploit a symmetry” and Chapter 6’s compression argument true in practice.

The inner product is a modeling choice, not a given. Because the covariance and therefore the principal components depend on the inner product, changing it changes the answer. If you weight the variables differently, standardize or not, or measure “distance” by something other than the plain sum of squared differences, the principal components move, because you have changed what “perpendicular” and “variance” mean. This is why two data scientists analyzing the same dataset, one on the raw features and one on PCA-transformed features, can run the same algorithm and get different results, and neither is simply “wrong”: they have made different structural assumptions about which basis and which inner product express the phenomenon’s real geometry. It is also why rotating a dataset before clustering can change the clusters: if the outcome depends on a rotation, the clustering algorithm is not using a rotation-invariant notion of distance, so it is quietly committed to a particular basis and inner product rather than to the intrinsic geometry of the points. The “right” inner product is the one aligned with the system’s own notion of similarity, and choosing it is a modeling decision you are always making, whether or not you notice.

There is a thread here worth naming, because it is one of your own. You have reasoned before that compression and the discovery of a system’s generating rules are related. PCA is the linear, statistical end of that idea: it finds the smallest basis that reconstructs the data well, and the number of large eigenvalues measures how compressible the data is. When only a few directions matter, the phenomenon is low-dimensional despite its many recorded features, and that low dimensionality is itself a form of structure, the same structure Chapter 6 argued a working model must be exploiting when it succeeds. In this linear setting, finding the right canvas and compressing the data are the same act: both amount to discovering the few directions along which the system actually lives. Statistical compression like PCA is the shallow end; knowing the generating rule would be the deep end, a point the curriculum returns to later.

A note on infinite dimensions. The function spaces of quantum mechanics are vector spaces too, but *infinite-dimensional*: no finite kit of building blocks spans them. “Infinite-dimensional” means exactly that any basis is infinite. There is still a basis and there are still coordinates; they just form an infinite list. For a space of functions, a natural basis is a family of standard waveforms (the sines and cosines of Fourier analysis are the archetype), and the coordinates of a function are its coefficients against that basis, the amount of each waveform it contains. This is the same move as before, a change to the canvas on which structure is visible, and it is why Chapter 5’s Fourier analysis is so powerful: it is a change of basis in an infinite-dimensional inner-product space, chosen so that the operation you care about becomes simple. An inner-product space complete enough to support this is called a Hilbert space, which is the setting your reading points to for the quantum-mechanical case.

Where you now stand

You came in using linear algebra as a set of procedures. You leave with the structure those procedures act on made explicit. A vector space is a type of structure, not a type of object: anything you can add and scale is one, which is why functions, quantum states, and gene-expression profiles are all vectors. A basis is a choice of description, so coordinates change while the vector does not, and a matrix is just the coordinate report of a linear map. Eigenvalues survive that arbitrariness, which is why they, not the matrix entries, carry the map’s intrinsic content. Your own intuition now has a precise home: linearity is a property of the system together with the canvas you represent it on, superposition made exact in the right coordinates, and choosing coordinates well is choosing the canvas where structure, often linearity, becomes visible. The inner product adds geometry, angle and length and orthogonality, and it too is a choice, which is why PCA, clustering, and every notion of “similarity” depend on a decision you are always making.

Notice what buying that geometry required: an inner product, and with it distances and angles, real numbers attached to pairs of vectors. That is a lot of structure, and it is fragile. Stretch the space and the distances change; the geometry you paid for is not preserved. So the next question is the one Klein’s hierarchy from Chapter 7 already pointed at from the top: what survives if you throw *all* measurement away, keeping neither length nor angle nor straightness, and retain only the crudest fact of which points are near which? That most permissive geometry, the structure that no amount of stretching or bending can destroy, is topology, the geometry that survives when every measurement is gone, and it is the subject of Chapter 9.